



TABLE OF CONTENTS

Introduction	1
Classes Overview	1
Heroes class	1
Dracula class	2
Sherlock Holmes class	2
Sisters class	2
Dr Watson class	2
InvisibleMan class	2
Fog class	2
Space class	3
Board class	3
Card class	3
CardResolver class	3
Player class	4
Json class	4
Ftxui_Front class	4
Raylib class	5
Controller class	6
References	6

1. INTRODUCTION

In the first phase of the project, the back-end structure and core game logic were implemented. As the central core of the system, this part was responsible for managing the game state, rules, cards, players, and logical interactions between various components, providing a stable and well-engineered foundation for the display layers. In the second phase, this foundation was extended with a new playable hero, the Invisible Man, together with his three Fog tokens; a JSON-based save/load system able to persist and restore a full game session to disk; and a brand-new graphical front-end built directly on top of the raylib library, which now serves as the game's main presentation layer in place of the original text-based terminal interface.

2. CLASSES OVERVIEW

Heroes class

This class serves as the base and core structure for defining and managing the attributes of various heroes and characters in the game, holding their key specifications.

Key Methods:

- **DrawnCard():** Simulates drawing a card from the deck. It first checks if the deck is empty; if so, it returns 0. Otherwise, it takes the top card, adds it to the player's hand, removes it from the deck, and returns 1.
- **Damage(int amount):** Applies damage to the hero. If the health drops to or below zero, it sets the hero's alive status to false, removes the hero from their current board space, and clears the space pointer; otherwise, it subtracts the damage amount from the health.
- **discard_hand():** Randomly selects a card from the player's hand, retrieves its boost value, moves the card to the discard pile, and removes it from the hand.
- **Discard_Card(Card *card):** Searches for a specific card pointer in the hand. If found, it moves the card to the discard pile and erases it from the hand; otherwise, it throws a runtime error.
- **toJson() & loadFromJson(...):** **PHASE 2** toJson() serializes the hero's full battle state (health, movement, action points, alive status, and the deck/hand/discard piles) into a Json object, while loadFromJson() restores that exact state from a saved Json object, using a callback to resolve hero-name references back into pointers.

Dracula class

The class associated with the Dracula character, implementing the specific logic, capabilities, and behaviors of this unique or villainous character during the game.

Sherlock Holmes class

This class is responsible for implementing the features, skills, and behaviors of Sherlock Holmes as one of the main and strategic characters in the game.

Key Methods & Initialization:

- **Sherlock Holmes Constructor:** Initializes Sherlock Holmes with core stats (name, melee attack type, 16 health, 2 movement) and populates its deck with character-specific and general cards (such as Administer Aid, Counterpunch, Feint, etc.). It then shuffles the deck randomly and draws the initial hand of 5 cards.

Sisters class

The class related to the sisters or related secondary characters, managing group interactions and specific rules regarding their presence in the gameplay.

Dr_Watson class

This class defines and controls the capabilities and logic related to Dr. Watson, guiding his supportive or instrumental role in the game.

Invisible Man class **PHASE 2**

A new playable hero added in the second phase of the project, representing the Invisible Man faction. Like the other heroes, it inherits from the Heroes base class and defines its own combat stats and a unique deck of cards built around stealth, misdirection, and fog manipulation.

Key Methods & Initialization:

- **InvisibleMan Constructor:** Initializes the hero with 15 health, a melee attack type, and 2 movement, then builds its full deck from stealth-themed cards (Coded Notes, Confound, Covert Preparation, Dreaming of Revenge, Emerge From Mist, Impossible to See, Into Thin Air, Lurking, Reign of Terror, Rolling Fog, Slip Away, Step Lightly, and Vanish), shuffles it, and draws an initial hand of 5 cards.
- **ability (Board &):** Reserved as the hero's special-ability hook, like every Heroes subclass; for the Invisible Man the actual ability logic is triggered and handled centrally through CardResolver and the Controller rather than inside the class itself.

Fog class **PHASE 2**

Represents a Fog token, one of three markers placed on the board that the Invisible Man uses to hide, teleport, and misdirect his opponent. It is implemented as a lightweight Heroes subclass so it can reuse the same positioning and board-occupancy mechanisms as an ordinary hero.

Key Methods:

- **Fog Constructor:** Creates a fog token identified by its number (1 to 3) with zero health and a fixed movement value of 2, since the token itself never fights or takes damage.
- **ability (Board &):** Left empty, as a fog token has no ability of its own; every effect involving it (moving it, teleporting a hero to or from it, etc.) is driven entirely by the Invisible Man's cards through CardResolver and the Raylib front-end.

Space class

This class models and manages the various spaces, tiles, or zones of the game board where pieces and cards are placed.

Key Methods:

- **Space Constructor:** Initializes a specific tile on the board by setting its unique identifier number, its associated zones (vector of spaces), and its neighboring tiles.
- **get_zone():** Returns a vector containing pointers to all space objects that share the same zone as this tile, which is essential for tracking ranged attacks and area effects.
- **get_neighbor():** Returns a vector of adjacent neighbor spaces connected to this tile, used primarily for movement and positioning logic.
- **get_hero() & set_hero (Heroes *hero):** Retrieves the pointer of the hero currently occupying this space or places/removes a hero on this tile during movement actions.
- **get_number():** Returns the unique integer identifier of the space.
- **get_Fog() & set_Fog (Heroes *) **PHASE 2**:** Retrieve or place one of the Invisible Man's fog tokens on this tile. The fog occupant is tracked separately from the ordinary hero occupant, so a single space can hold both a hero and a fog marker at the same time.
- **add_portal (Space *), get_portal() & has_portal() **PHASE 2**:** Model hidden passages between non-adjacent tiles: add_portal() links this space to a destination space, while get_portal() and has_portal() are used when resolving movement and fog-token effects that can jump through these hidden ways.

Board class

Represents the main game board and takes responsibility for managing tile layouts, player positions, and the overall status of the board structure.

Key Methods & Implementation:

- **Board Constructor:** Initializes the game board by resizing the spaces vector to 32 tiles and defining the precise connectivity graph, including zone associations and immediate neighbors for each individual space.
- **is_Adjacent (Space *s1, Space *s2):** Checks whether two given space pointers are directly adjacent to each other by iterating through the neighbor list of the first space, returning true if a connection exists.
- **get_spaces():** Returns a reference to the entire collection of spaces comprising the board structure.
- **SwapHeroes (Heroes *hero1, Heroes *hero2):** Manages tactical repositioning by safely swapping the positions and associated board space pointers of two heroes, throwing runtime errors if either hero is invalid or unplaced.

Card class

The base class for defining game cards, holding attributes such as card type, effects, and information related to each card.

CardResolver class

This class is responsible for interpreting, processing, and executing card effects after they are played by players, handling the card rules logic.

Key Methods & Interpretations:

- **execute():** This voluminous and key method uses a large switch-case structure on the card type (CardType) to examine what card has been played. Then, depending on which player or hero the card belongs to (such as Dracula, Sherlock Holmes, Sisters, or Dr. Watson), it applies the specific effects of that card; effects such as modifying attack and defense values (Attack_Value and Defence_Value), moving heroes on the board, applying or reducing damage and health, drawing a card from the hand (DrawnCard), or disabling opponent capabilities.
- **TryDisableCard():** This auxiliary method is designed for managing feint and neutralization cards (such as Feint). It first checks whether a valid card has been passed or not. Then it checks if the card owner is Sherlock Holmes or Dr. Watson and Sherlock is alive, it protects the card from being disabled; otherwise, by calling setApplyEffects(false), it neutralizes and invalidates the opponent card effects.
- **execute() PHASE 2:** The switch-case was extended with a full set of cases for the Invisible Man's thirteen card types (Coded Notes, Confound, Covert Preparation, Dreaming of Revenge, Emerge From Mist, Impossible to See, Into Thin Air, Lurking, Reign of Terror, Rolling Fog, Slip Away, Step Lightly, and Vanish). These cases resolve fog-token movement and placement through the Raylib front-end's fog-token menus, grant the Invisible Man bonus defence or damage while he stands on a fog token, and let him teleport onto or away from a token space.

Player class

Manages and maintains information regarding players, scores, cards in hand, and each user's status throughout the game.

Key Methods:

- **set_count(int) & set_isAttacker (bool) PHASE 2:** Absolute setters added alongside the existing incremental add_count(), so the exact action count and attacker/defender role of a player can be restored precisely when a saved game is loaded.

Json class PHASE 2

A small, self-contained JSON value type and parser/serializer, added as the foundation of the save/load system. A single Json object can hold a null, boolean, number, string, array, or object value, and the class provides everything needed to build such a tree in code and convert it to and from JSON text, without relying on any external library.

Key Methods:

- **makeArray() & makeObject():** Static factory methods that return an empty Json value already tagged as an array or an object, used as the starting point before calling push_back() or set().

Ftxui_Front class

This class manages the interaction with the FTXUI-based presentation layer, rendering a text-based and engaging user interface in the terminal environment.

Key Methods & Interpretations:

- **Menu():** Creates an interactive full-screen menu for the main game screen (including game, instructions, and exit options). It uses the FTXUI library to process keyboard events (such as the Enter key) and returns the appropriate `e_Menu` enumeration value based on the user's choice.
- **Players_Info_List (Player *p1, Player *p2):** Implements a two-column form user interface for entering player information (name and age) along with a confirmation button. Upon user confirmation, it directly sets and saves the entered information onto the player objects.
- **Det_characters (Player p1, Player p2):** Compares the players' ages so that the younger player gets priority in choosing a hero, then displays a menu for selecting a character (such as Dracula or Sherlock Holmes) and returns the final selection.
- **catch_place (Player *p1, Player *p2, Board board):** Receives the initial placement location of the heroes on the game board through a graphical menu and, considering the players' ages, assigns board positions (tiles number 3 and 23) to the heroes.
- **Graph_Box(vector<Space> &spaces):** Renders the complete map and board graph inside a Canvas. This method visually and dynamically renders the connecting lines between spaces and the placement status of heroes on each tile (along with colors and brief info) in the terminal.
- **choose_comrad_place (Player p1, Player p2, Board board):** Manages the manual positioning of sidekicks and assistants for each player (the three sisters for Dracula and Dr. Watson for Sherlock) within permitted zones on the board and places them in the chosen tile after confirmation.
- **Attakcer_Heroes_Menu (Player *player, Board board, Heroes *&hero):** Extracts the list of living combatants belonging to the player (including the main hero and sidekicks) and provides a menu to select the current attacker during the turn.
- **Defender_Heroes_Menu(...):** Filters eligible opposing defenders based on the attack type (melee or ranged) and by checking paths and tile adjacency, then renders the defender selection menu for the user.
- **Attacker_selected_card(...):** Filters cards in the attacker's hand that are compatible with the attack type, displays the attack card selection menu, and finalizes the chosen card.
- **Defender_selected_card(...):** Suitable defense cards from the defender's hand (or associated sidekicks) are extracted, providing the option for a "no defense" choice, and applying the selected defense card.
- **MoveHero (Heroes hero, Board board, int max_distance):** Uses graph search algorithms (BFS) to find all reachable tiles up to a specified distance from the hero's current location and presents a destination selection menu to the user.
- **SelectHero (Board board):** Scans and gathers all living combatants present on the game board and provides an interactive menu to select one of them for various operations.
- **PlaceHeroAdjacent (Heroes hero, Heroes target, Board *board):** Finds permitted empty adjacent tiles around a specific target and allows the user to place the desired hero in those surrounding positions.
- **DiscardCards (Heroes dracula):** Allows the player to interactively select and discard specific cards from a hero's hand (such as Dracula) to complete the card discarding process.
- **Event_Selected_Card(...):** Filters event-type cards from the player's hand, displays the event card selection interface, and prepares the card to execute special effects.

Raylib class **PHASE 2**

Introduced in the second phase, this class replaces the terminal-based `Ftxui_Front` as the project's active presentation layer, rendering the game as a full graphical, mouse-driven application built directly on top of the raylib graphics library (windows, textures, a custom font, and clickable rectangles, instead of a text UI). It mirrors almost the entire method surface of `Ftxui_Front`—the same menus, hero and card selection dialogs, and combat flow—but implemented with drawn buttons, hero cards and an interactive board map, and it additionally introduces the fog-token interactions needed for the Invisible Man.

Key Methods & Interpretations:

- **GetGameFont() & UnloadGameFont():** Lazily load a custom font the first time it is needed and cache it for reuse across every screen, releasing it once when the window is closed.
- **Menu_():** Draws the graphical main menu (Play, Continue, Help, Exit) as clickable buttons and returns the corresponding `e_Menu` value once the player selects an option with the mouse or keyboard.
- **Players_Info_List(...) & Det_characters(...):** Render on-screen text boxes so both players can type their name and age, then, after comparing ages to decide who chooses first, display clickable character portraits (Dracula, Sherlock Holmes, Invisible Man, ...) and return the chosen character indices.
- **catch_place(...) & choose_comrad_place(...):** Let the players click their hero's and comrades' starting tiles (sisters, Watson, fog tokens) directly on the board map, respecting each faction's allowed starting zones.
- **DrawBoardMap (Board, Rectangle) & DrawHeroBox (Player, Rectangle):** Core rendering helpers; `DrawBoardMap` paints every tile, connecting line, hero token and fog token of the board inside a given screen rectangle, while `DrawHeroBox` draws a player's portrait, health and action counters inside a fixed info panel.
- **main_map (Player, Player, Board, Player):** Combines `DrawBoardMap` and `DrawHeroBox` with the turn indicator and action buttons into the main gameplay screen shown every turn.
- **Attakcer_Heroes_Menu(...) & Defender_Heroes_Menu(...):** Graphical equivalents of the `Ftxui` menus; they highlight the clickable combatants directly on the board itself instead of showing a text list, filtering eligible defenders by attack type and tile adjacency exactly as before.
- **Attacker_selected_card(...), Defender_selected_card(...) & Reveal_Combat(...):** Show the attacker's and defender's hands as clickable card sprites, let each side pick a card, then animate a face-off screen revealing both chosen cards before combat is resolved.
- **MoveHero (Heroes, Board, int, Player, Player):** Runs a breadth-first search over the board graph to highlight every tile reachable within the given distance, and moves the hero to whichever highlighted tile the player clicks.
- **SlotMenu (const vector<string> &, const string &):** A reusable graphical list-of-buttons widget, with one button per save slot, used by both the Continue Game screen and the in-game Save Game screen.
- **SelectFogToken(...), MoveFogToken(...), MoveFogTokenDistance(...), MoveFogTokenAnywhere(...), MoveFogTokenToEmptySpace(...), MoveHeroToFogSpace(...), ChooseBetweenTwoEffects(...), SelectAdjacentHero(...) & PlaceHeroOnBoard(...):** A family of methods added specifically for the Invisible Man's cards; together they let the player click one of the three fog tokens, move it a fixed or unlimited distance, teleport the Invisible Man onto or off a token, or choose between two possible resolutions for cards such as Lurking.
- **DeclareWinner (Heroes *) & Show_Help():** Display the end-of-game victory screen for the winning hero, and a scrollable in-game help/rules screen, respectively.

Controller class

The central controller of the game that directs program execution flow, receives inputs, updates states, and coordinates between various back-end and front-end components.

Key Methods & Interpretations:

- **set_players_character (int choise)**: Assigns character roles (Dracula, Sherlock Holmes, and their companions) to the players based on the user's choice and the players' ages, determining who plays which faction.
- **get_younger_player() & get_older_player()**: Compares the ages of both players (p1 and p2) to identify and return a pointer to either the younger player (user priority) or older player.
- **Initial_turn()**: Sets the initial turn of the game by designating the younger player as the active turn player and the other as the opponent.
- **change_turn(), run() ...**: Alternates the active turn and runs the central game loop and execution engine.
- **GameLoop() PHASE 2**: Added as the private core loop that run() enters once a match has been set up or a saved game has been loaded; it repeatedly drives attacker/defender selection, resolves cards through CardResolver, and alternates turns using the Raylib front-end (RF), until the match ends or the player chooses to save and exit.
- **SaveGame (int slot) & LoadGame (int slot) PHASE 2**: Persist a full game session to disk. SaveGame walks every hero, the board's tile and fog-token occupancy, both players, and the current turn/combat state into a Json tree (using each hero's toJson() and a couple of helper lambdas) and writes it as text to a saves/slot_<n>.json file; LoadGame resets every game object to a clean state and then reverses the process, restoring hero stats, decks/hands, board occupancy and any in-progress combat from the parsed Json.
- **EncodeCardRef (Card *) & DecodeCardRef (const Json &) PHASE 2**: A helper pair used only by the save system. Because a Card normally points inside a hero's own hand vector, EncodeCardRef converts such a pointer into a portable (hero, index) reference before saving, and DecodeCardRef looks the card back up inside the correct hero's hand after loading.
- **HasSave (int slot) & GetSaveSummary(int slot) PHASE 2**: Check whether a given save slot's file exists and produce a short, human-readable summary of it, used to label the slot buttons on the Continue Game and Save Game screens.

3. REFERENCES

- <https://cppreference.com/>
- C++ How to Program Book
- <https://github.com/ArthurSonzogni/FTXUI.git>
- <https://github.com/>
- <https://claude.ai/>
- <https://github.com/raysan5/raylib>